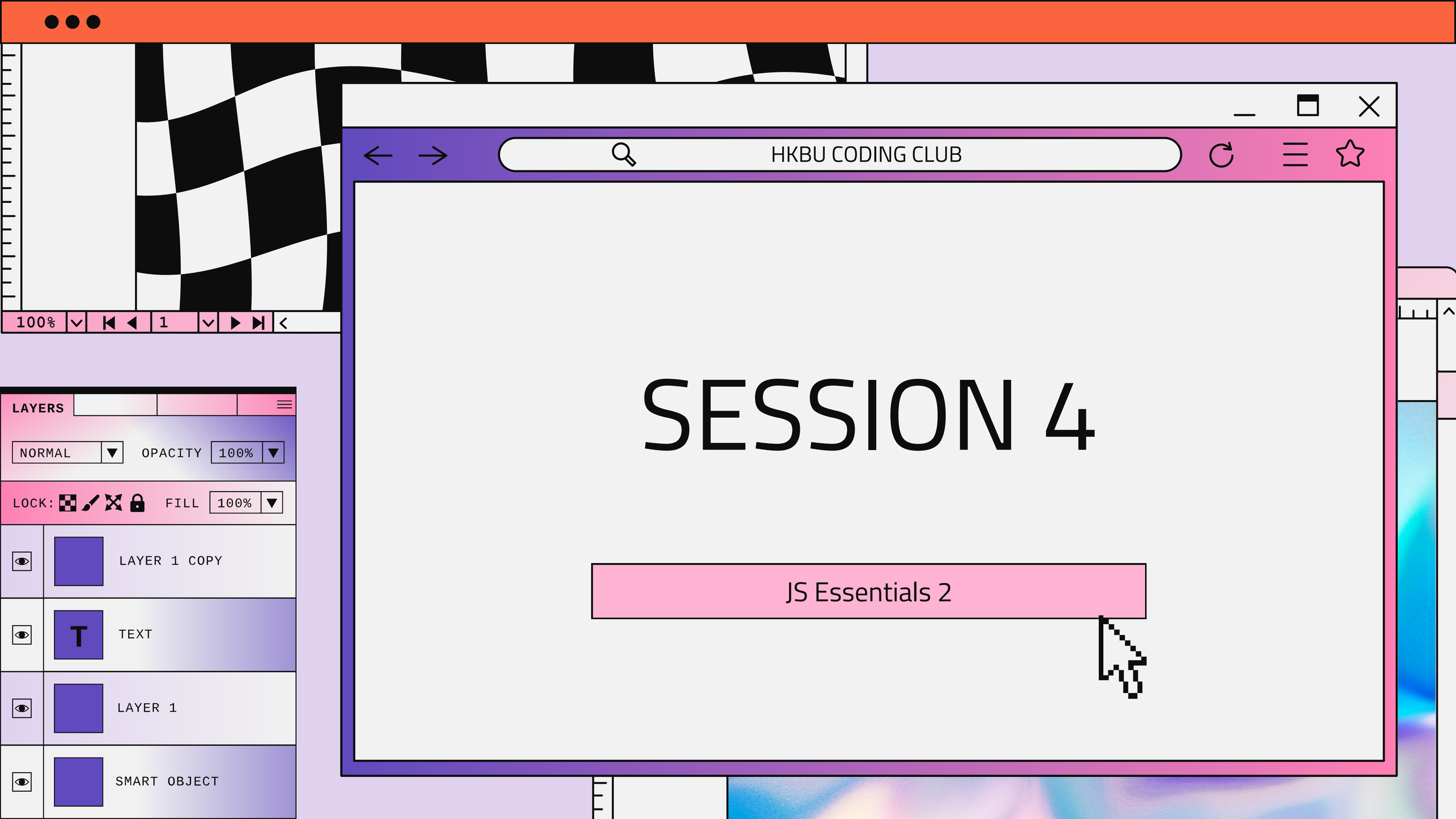

HKBU CODING CLUB
SESSION 4
JS Essentials 2
100%
1
LAYERS
NORMAL
OPACITY
100%
LOCK:
FILL
100%
LAYER 1 COPY
T
TEXT
LAYER 1
SMART OBJECT

WORKSHOP PLAN

img000.jpg



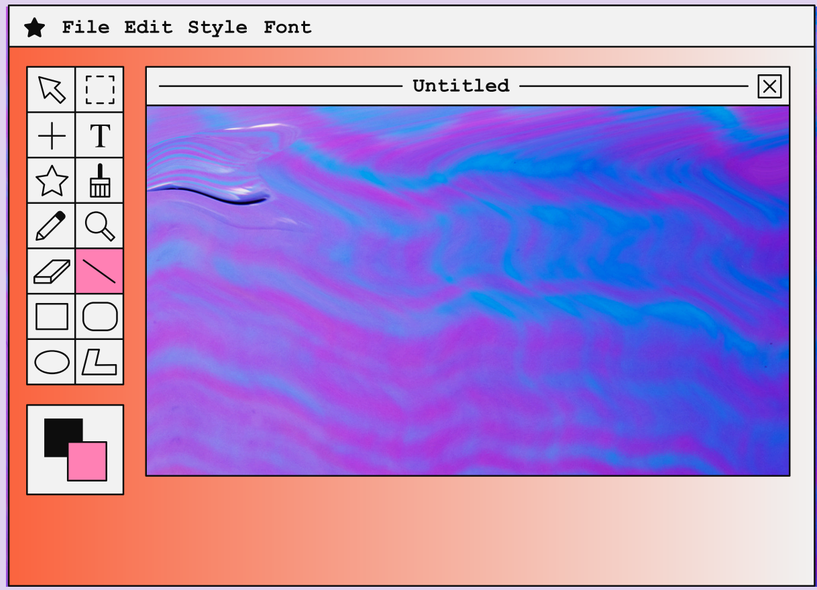
Untitled - TextEdit

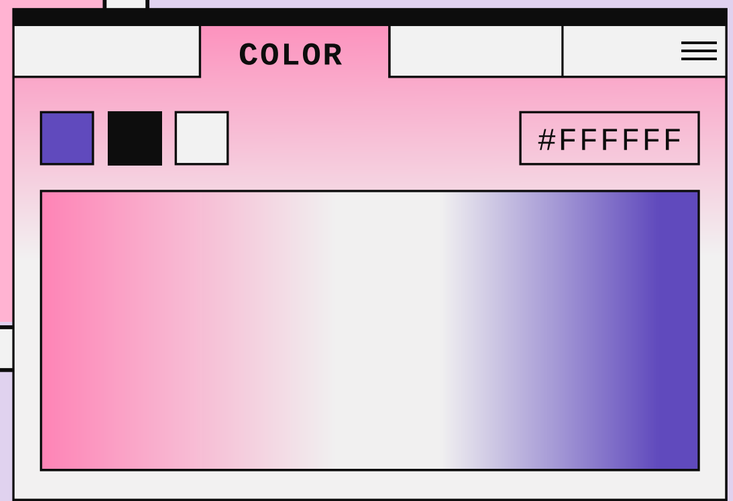
File Edit View Help

01	**INTRODUCTION TO HTML & CSS**
02	**Creating Modern Websites using Bootstrap and Gen-AI**
03	**JS Essentials 1**
04	**JS Essentials 2**
05	**Hike**
06	**Introduction to Git and Deployment**

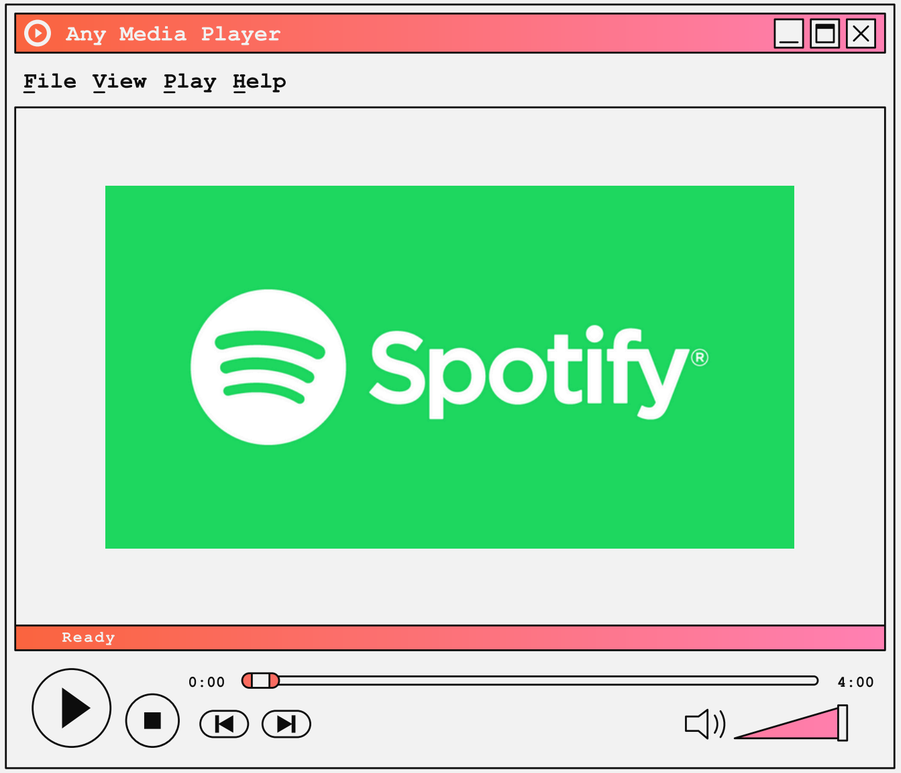
JS Essentials 2

HKBU CODING CLUB





ANALOGY



Play Button

Stop Button

Prev Button

Next Button

How it works?

Some JS Behind this

How to do it?

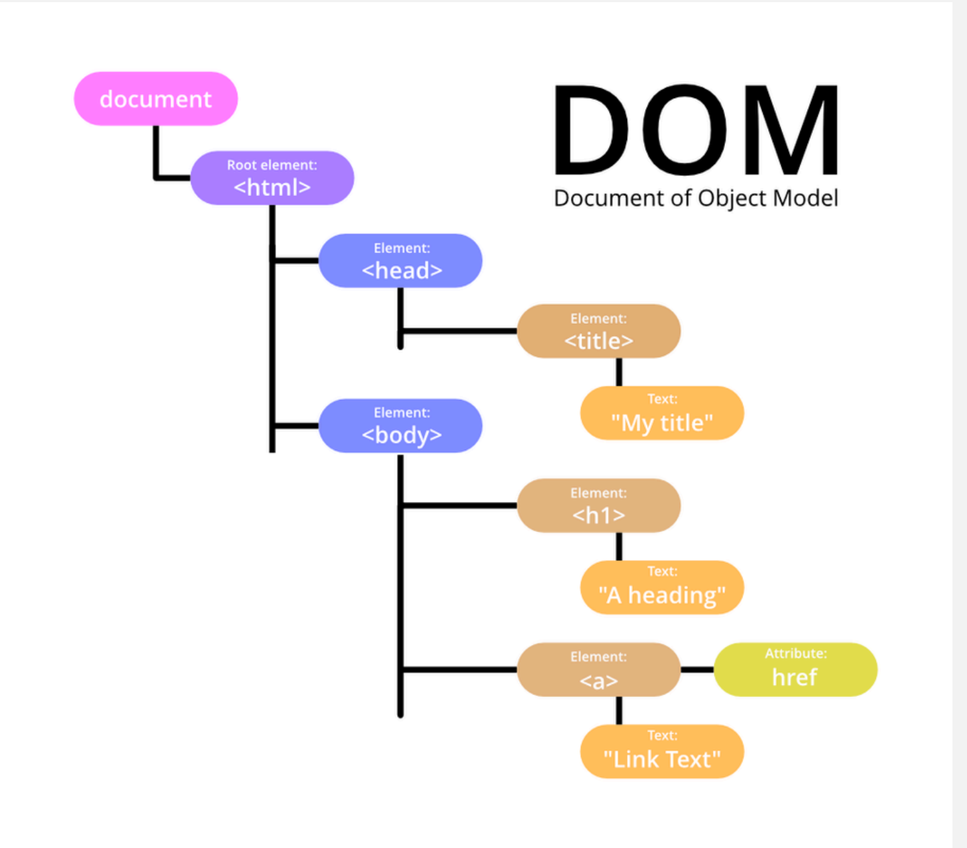
Let’s do this today!

What is DOM?

- The Document Object Model, usually referred to as the DOM, is an essential part of making websites interactive.
- It is an interface that allows a programming language to manipulate the content, structure, and style of a website.
- JavaScript is the client-side scripting language that connects to the DOM in an internet browser.

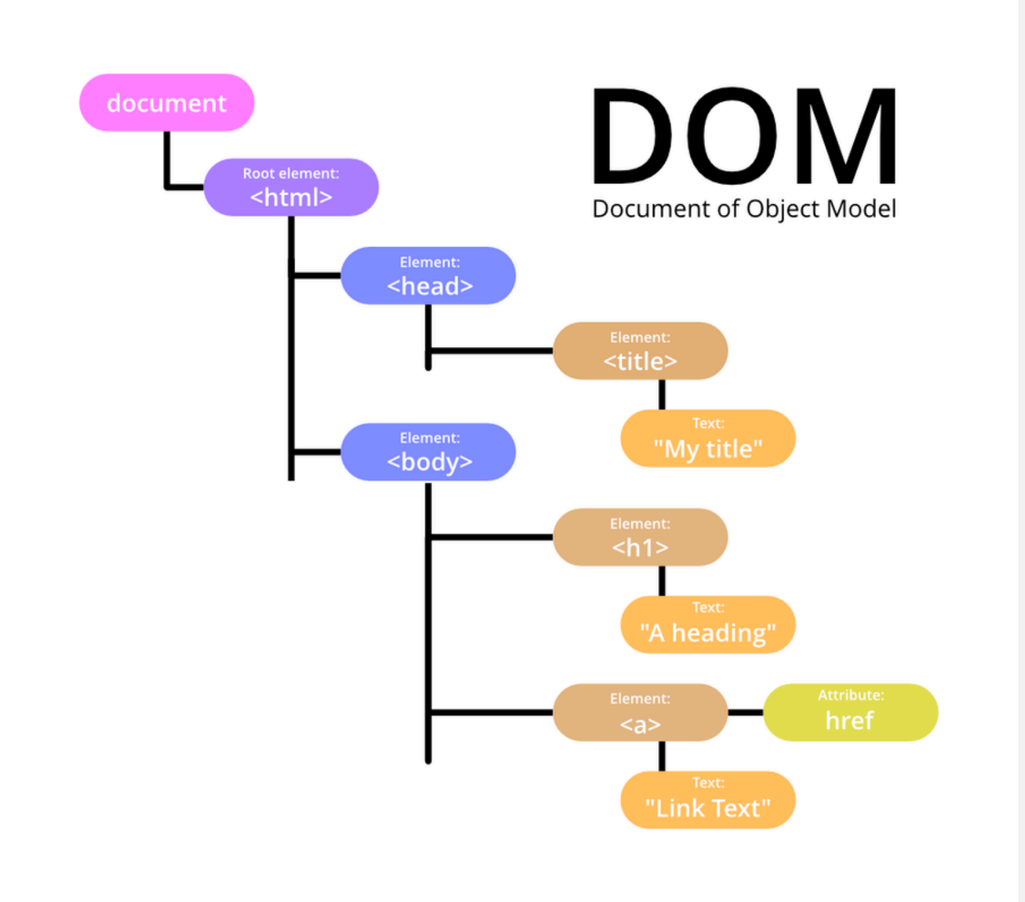
DOM

- The browser creates a representation of the document known as the Document Object Model. This model allows JavaScript to access the text content and elements of the website document as objects.



- Document is an object, body is a property of that object that we have accessed with dot notation. Submitting document.body outputs the body element and everything inside of it.

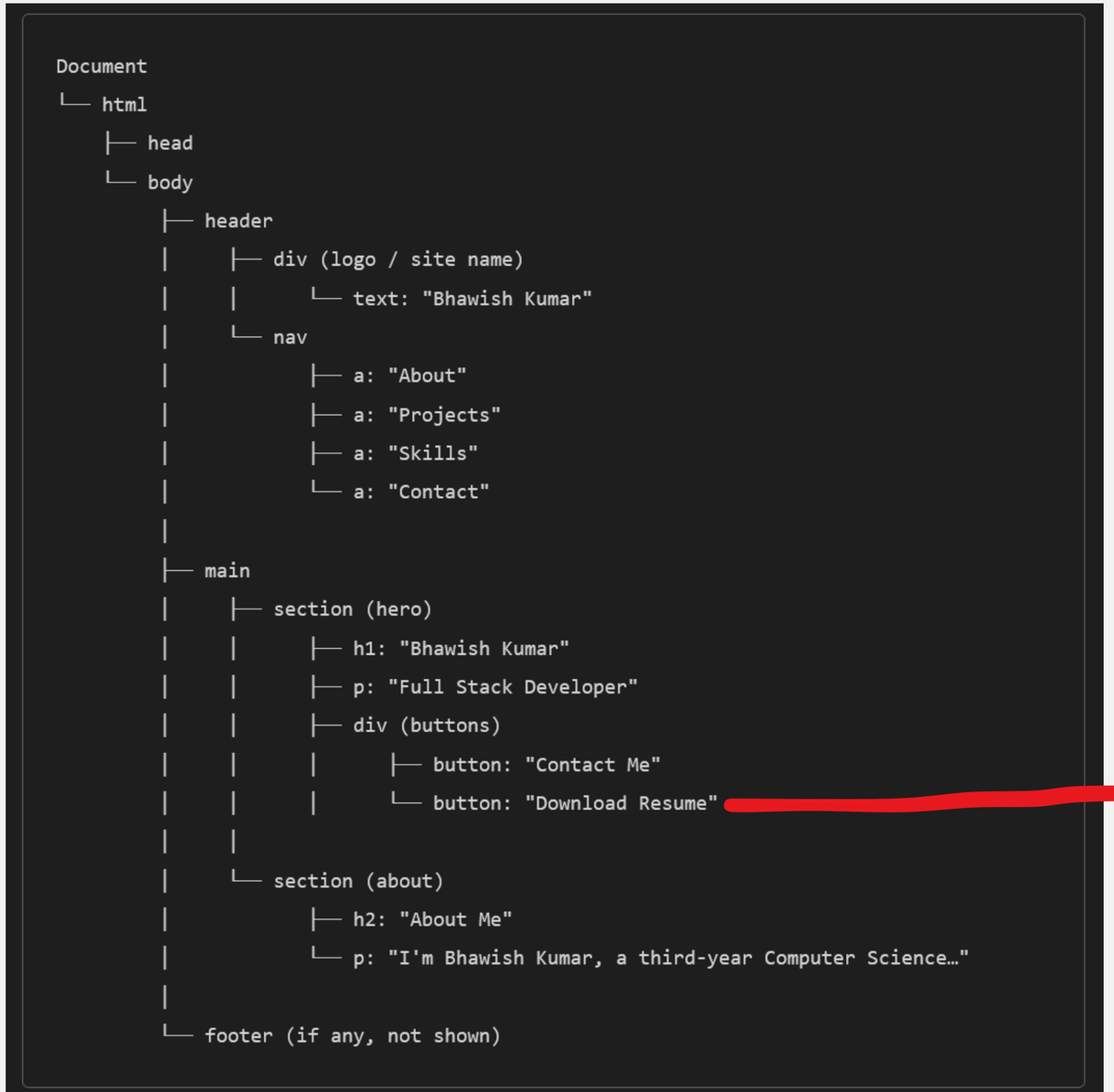
- Now can change some of the live properties of the body object on this website. Example: document.body.style.backgroundColor = 'fuchsia';



DOM

- DOM can be studied more in-depth as well but we don’t have the time for that here.
- The material was taken from this book.

DOM IN OUR WEBSITE



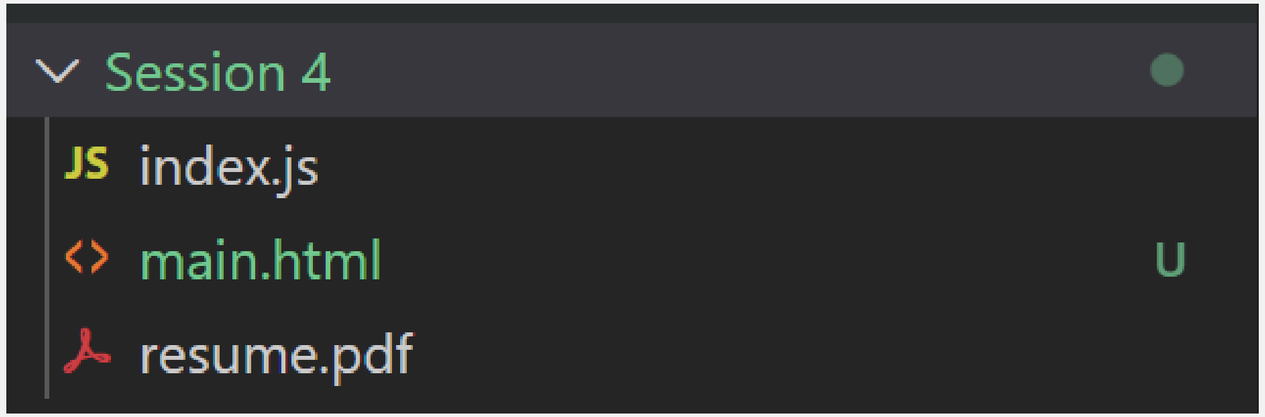
HOW?

HAVE SOME FUNCTIONALITY

FOLLOW ME!!!

1) Create index.js in same folder as your main.html

2) Put your Resume in the same folder as your main.html



FOLLOW ME!!!

3) Connect index.js to main.html

AT THE END OF OUR BODY PLACE:

```
<script src="index.js"></script>
```

FOLLOW ME!!!

3) Connect index.js to main.html

AT THE END OF OUR BODY PLACE:

```
<script src="index.js"></script>
```

FOLLOW ME!!!

remmeber class and id from last session!

3) Give your button an id

```
<a  class="btn ghost" id="download-resume"
    download>Download Resume</a>
```

FOLLOW ME!!!

remmeber class and id from last session!

4) Give your button an id

```
<a  class="btn ghost" id="download-resume"
    download>Download Resume</a>
```

Index.js

```
// 1) Get the button
const btn = document.getElementById("download-resume");

// 2) When clicked...
btn.addEventListener("click", () => {
 // 3) Create a temporary link
 const link = document.createElement("a");
 link.href = "resume.pdf";       // path to your PDF
 link.download = "Bhawish_Kumar.pdf";   // save-as filename

 // 4) Trigger a synthetic click
 link.click();
});
```



The browser has already built the DOM tree of the webpage.

We are asking the DOM:

“Please find the element whose id is download-resume.”

resume button

becomes a reference to that element in JavaScript memory.

Index.js

```
// 1) Get the button
const btn = document.getElementById("download-resume");

// 2) When clicked...
btn.addEventListener("click", () => {

 // 3) Create a temporary link
 const link = document.createElement("a");
 link.href = "resume.pdf";       // path to your PDF
 link.download = "Bhawish_Kumar.pdf";   // save-as filename

 // 4) Trigger a synthetic click
 link.click();
});
```



We are telling the browser: “When the user clicks this button, run the following code.” addEventListener attaches behavior to an element. This separates HTML structure from JavaScript behavior → best practice.

Index.js

```
// 1) Get the button
const btn = document.getElementById("download-resume");

// 2) When clicked...
btn.addEventListener("click", () => {

 // 3) Create a temporary link
 const link = document.createElement("a");
 link.href = "resume.pdf";       // path to your PDF
 link.download = "Bhawish_Kumar.pdf";   // save-as filename

 // 4) Trigger a synthetic click
 link.click();
});
```



JavaScript can create new DOM elements dynamically. This does not add it to the page visually.
You are generating an <a> tag behind the scenes.

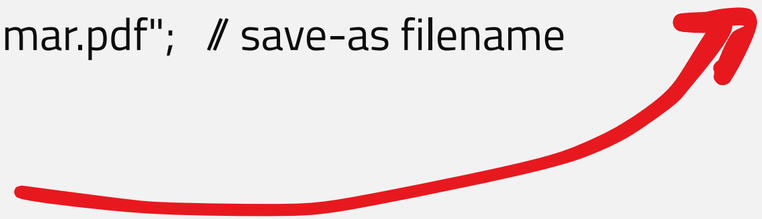
Index.js

```
// 1) Get the button
const btn = document.getElementById("download-resume");

// 2) When clicked...
btn.addEventListener("click", () => {

 // 3) Create a temporary link
 const link = document.createElement("a");
 link.href = "resume.pdf";       // path to your PDF
 link.download = "Bhawish_Kumar.pdf";   // save-as filename

 // 4) Trigger a synthetic click
 link.click();
});
```



- **You are simulating a real user clicking the hidden link.**
- **The browser treats this exactly like a real click.**
- **This is what actually starts the download.**

DOM Manipulation

DEFINITION

DOM manipulation is the process of using JavaScript to dynamically add, remove, and modify the content, structure, and style of a webpage

EXAMPLES

- element.textContent
- element.innerHTML
- element.style
- element.classList.add/remove/toggle

PROJECT SECTION

USING DOM MANIPULATION

PUT IN BODY!

```
<!-- Projects Section -->


  <section class="projects" id="projects">
    <h2>Projects</h2>
 <article class="project-card">
  <h3 class="project-title">Weather App</h3>
  <p class="project-teaser">A minimal app that shows real-time weather.</p>

  <button class="toggle-details"
          aria-expanded="false"
          aria-controls="proj-1-details">
    More details
  </button>

  <div class="project-details" id="proj-1-details" hidden>
    <ul>
      <li><strong>Stack:</strong> HTML, CSS, JS, OpenWeather API</li>
      <li><strong>Features:</strong> Search by city, geolocation</li>
      <li><strong>Repo:</strong> <a href="#">github.com/you/weather-app</a></li>
      <li><strong>Live:</strong> <a href="#">yourdomain.com/weather</a></li>
    </ul>
  </div>
 </article>

  <!-- Duplicate the same structure for more cards, just change IDs -->
</section>
```

PUT IN STYLE

```
.projects{
 max-width: 920px;
 margin: 32px auto;
 padding: 0 16px;
 }
 .project-card {
 background: #0f172a; /* dark bg */
 border: 1px solid #1f2937; /* subtle border */
 border-radius: 12px;
 padding: 16px 20px;
 color: #e5e7eb;
 max-width: 640px;
 margin: 32px auto;
 }
 .toggle-details {
 margin-top: 8px;
 padding: 8px 12px;
 border-radius: 8px;
 border: 1px solid #334155;
 background: #0b5ed7; /* blue button */
 color: white;
 cursor: pointer;
 }
```

```
.toggle-details[aria-expanded="true"] {
  background: #0a58ca;
}

.project-details {
  overflow: hidden;
  transition: grid-template-rows 250ms ease, opacity 250ms ease;
  display: grid;
  grid-template-rows: 0fr; /* collapsed */
  opacity: 0;
  margin-top: 8px;
}

.project-details > * {
  min-height: 0; /* required for the grid trick */
}

.project-details.is-open {
  grid-template-rows: 1fr; /* expanded */
  opacity: 1;
}

.project-details[hidden] {
  display: none; /* real hidden for screen readers until opened first time */
}
```

INDEX.JS

```
// 1) Find all buttons with class "toggle-details"
var toggles = document.getElementsByClassName('toggle-details');

// 2) Attach a click handler to each button
function attachToggleHandlers() {
  for (var i = 0; i < toggles.length; i++) {
    var button = toggles[i];
    // Use a named function so it's easier to read and reuse
    button.addEventListener('click', onToggleClick, false);
  }
}
```

```
// 3) When a toggle button is clicked...
function onToggleClick(event) {
 var button = event.currentTarget;

 // 3a) Find the associated panel using aria-controls
 var panelId = button.getAttribute('aria-controls');
 var panel = document.getElementById(panelId);

 // 3b) Read current state explicitly as a string
 var currentExpanded = button.getAttribute('aria-expanded'); // "true" or "false"
 var isExpanded = (currentExpanded === 'true');

 // 3c) Decide the next state
 var nextExpanded;
 if (isExpanded === true) {
  nextExpanded = 'false';
 } else {
  nextExpanded = 'true';
 }

 // 3d) Update the button's ARIA state
 button.setAttribute('aria-expanded', nextExpanded);
 // 3e) Show or hide the panel using the 'hidden' property
 if (nextExpanded === 'true') {
 // Opening
 panel.hidden = false;
 // Add a class to let CSS animate if you want, but it's optional
 addClass(panel, 'is-open');
 } else {
 // Closing
 removeClass(panel, 'is-open');
 // Hide the panel directly (keeps screen readers from announcing it)
 panel.hidden = true;
 }
 }
```

```
// 4) Small helper functions (long-form versions)
function addClass(element, className) {
  // Avoid short-cuts; use classList in a direct, readable way
  element.classList.add(className);
}

function removeClass(element, className) {
  element.classList.remove(className);
}

// 5) Start everything
attachToggleHandlers();
```

WHAT YOU SHOULD DO NOW?

CONTACT SECTION

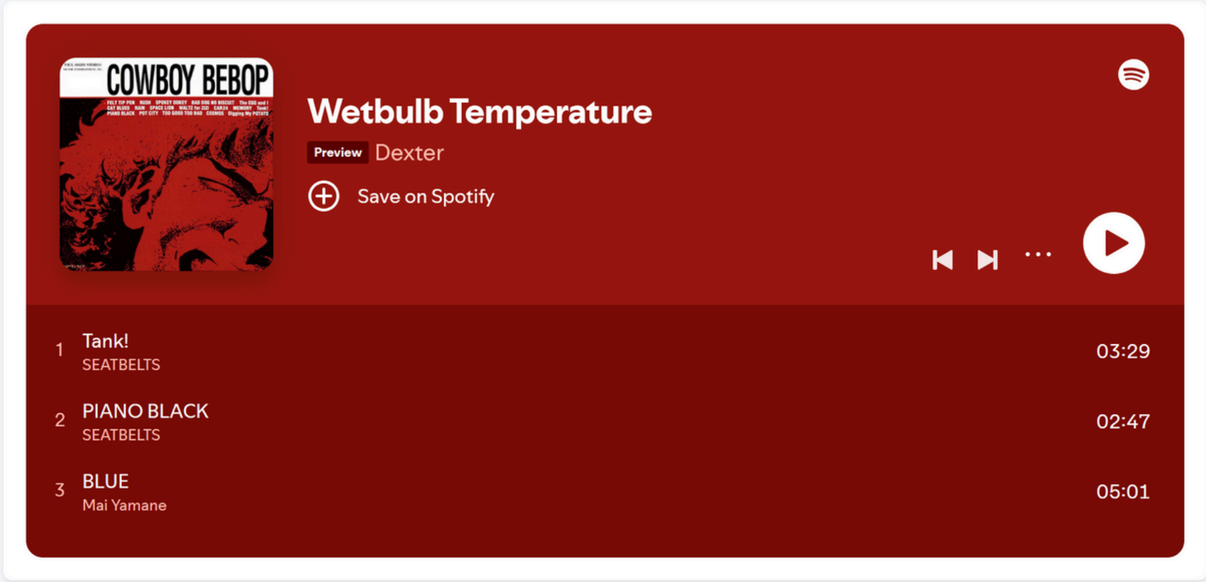
- SIMPLE: Design
- DIFFICULT: USE FORMS

EXTRAS

HOW TO ADD SPOTIFY?

Things I Do for Fun

Music keeps me focused while coding. Here's what's usually playing in the background.



iFrame

An iframe (inline frame) is
an HTML element that embeds another HTML document, essentially creating a "window" to display content from a different source (like a YouTube video, Google Map, or social media feed) directly within your webpage without the user leaving your site



src="https://open.spotify.com/embed/
playlist/YOUR_PLAYLIST NAME "

iFrame

```
<div class="card shadow-sm border-0 p-3">
  <iframe
    id="spotifyPlayer"
    src="https://open.spotify.com/embed/playlist/FOCUS_PLAYLIST_ID"
    width="100%"
    height="380"
    frameborder="0"
    allow="encrypted-media"
    style="border-radius:12px;">
  </iframe>
 </div>
</section>
```

Microsoft Copilot Chat

